

National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab

(CL -2001)

Course Instructor	Mr. Muhammad Naveed
Lab Instructor	Mr. Durraiz Waseem
Lab Demonstrator	Ms. Adeela Nasir
Section	BDS-3A
Semester	Fall 2025

*Department of Computer Science
FAST-NU, Lahore, Pakistan*

Lab Manual 11

Objectives:

- Understand the concept and implementation of Heap
- Learn how to solve difficult level problems on Heap

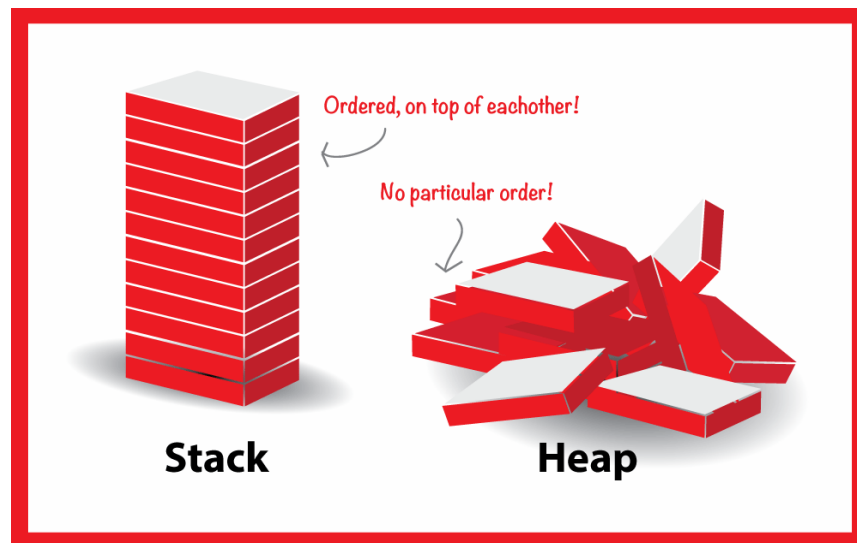
In the world of data structures, **Heaps** serve as a foundation for efficient **priority-based operations**, such as scheduling, shortest-path algorithms, and efficient sorting (like **Heap Sort**).

Unlike Binary Search Trees (BSTs), Heaps do **not maintain a sorted order**, but they guarantee quick access to the **highest (or lowest) priority element**.

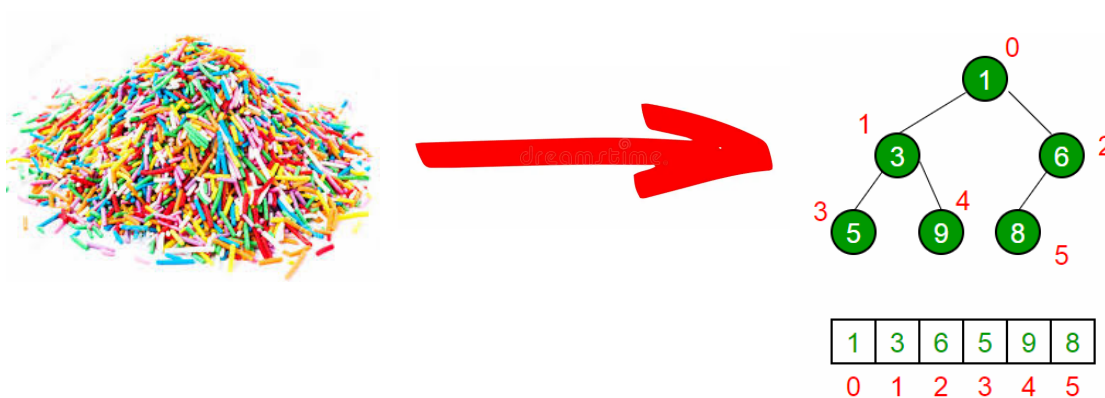
The Need for Heaps

While **Stacks** and **Queues** manage data in a specific order (LIFO/FIFO), they don't support **priority management**, every element is treated equally.

In contrast, a **Heap** ensures that the **most important element (min or max)** is always easily accessible.



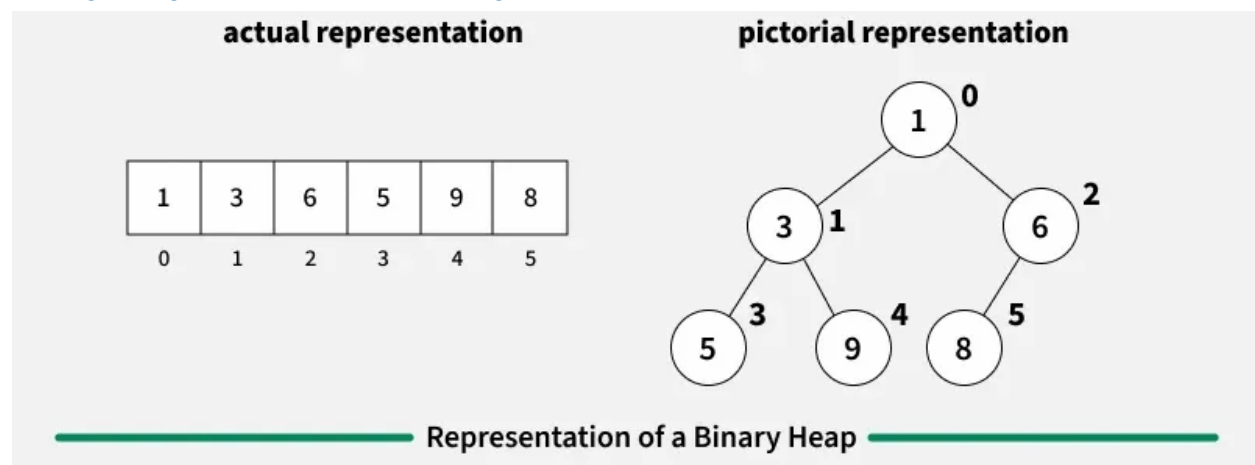
Heap



A **Heap** is a **complete binary tree**, meaning all levels are filled except possibly the last, which is filled from left to right. It satisfies one of the following **ordering properties**:

Type	Definition
Max-Heap	Each parent node has a value greater than or equal to its children.
Min-Heap	Each parent node has a value less than or equal to its children.

Heap Representation & Operations



Heaps are commonly implemented as arrays (not linked nodes like trees).

For an element at index i :

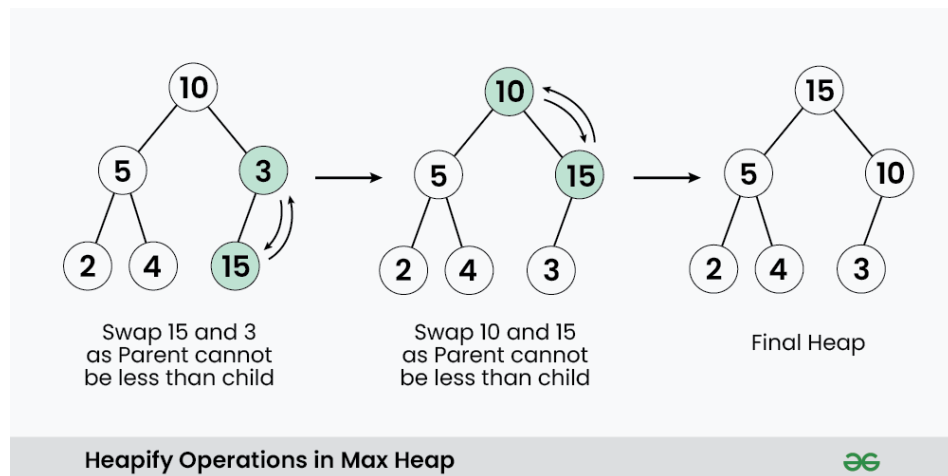
- Parent = $\lfloor i/2 \rfloor$
- Left Child = $2i$
- Right Child = $2i + 1$

This representation ensures $O(1)$ access to parent and child nodes without pointer overhead.

Operation	Description	Time Complexity

Insert	Add a new element at the end, then "heapify-up" to restore order	$O(\log n)$
Extract (Min/Max)	Remove the root element, move the last node to root, and "heapify-down"	$O(\log n)$
Peek (Min/Max)	Return the root element without removing it	$O(1)$
Heapify	Convert an unsorted array into a heap	$O(n)$
Delete	Remove a specific node and re-heapify	$O(\log n)$

Heapify Process



Heapify-Up (Insertion)

Used after inserting an element at the end:

- Compare the inserted node with its parent.
- Swap if the Heap property is violated.
- Continue until the property is restored.

Heapify-Down (Deletion / Extraction)

Used after removing the root:

- Move the last element to the root.
- Compare it with its children.
- Swap with the child that violates the heap property until balance is restored.

Heap Variants

Type	Description	Use Case
Binary Heap	Each node has ≤ 2 children	General-purpose priority queue
Binomial Heap	Collection of binomial trees	Mergeable heaps
Fibonacci Heap	Uses lazy merging, very efficient amortized times	Dijkstra's / Prim's Algorithm
Min/Max Heap	Specialized for smallest/largest element access	Scheduling, event simulation

Structure and Properties

Term	Description
Node	Basic element containing value and references
Complete Tree	All levels are full except possibly the last

Heap Property	Maintains parent-child ordering
Heapify	Process of maintaining heap property
Root	Always holds the highest (max-heap) or lowest (min-heap) value

Comparison with BSTs

Property	BST	Heap
Order Property	Left < Root < Right	Parent $\geq$ (or $\leq$) Children
Shape Property	Can be unbalanced	Always complete
Search	$O(\log n)$ average	$O(n)$
Insert/Delete	$O(\log n)$	$O(\log n)$
Find Min/Max	$O(\log n)$ (depends on tree shape)	$O(1)$
Use Case	Searching, sorting	Priority management

Advantages

- Guaranteed $O(\log n)$ insertion and deletion
- Fast access to highest/lowest element ($O(1)$)
- Memory-efficient array-based representation

- Useful for scheduling and real-time systems

Limitations

- No fast search for arbitrary elements ($O(n)$)
- Not suited for ordered traversal (unlike BSTs)
- Re-heapifying cost during frequent updates



Background

In the world of **Bleach**, the balance between the **Soul Society**, **Hueco Mundo**, and the **World of the Living** is delicate, just like the structure of a **Heap**, where order must always be maintained. Head Captain **Genryūsai Yamamoto** has entrusted you, a rising **Shinigami Data Engineer**, with constructing the **Soul Heap**, a system that efficiently ranks and retrieves the strongest warriors (Soul Reapers, Arrancars, or Hollows) based on their **spiritual pressure (reiatsu)**.

Unlike BSTs or AVL Trees, a **Heap** focuses solely on hierarchy, it doesn't care about sorted order across all nodes, only that **each parent dominates (or submits to)** its children depending on whether it's a **Max Heap** or a **Min Heap**.

Your task is to ensure this spiritual hierarchy remains intact after every insertion or deletion, just as Yamamoto enforces order after every battle.

In-Lab Tasks

STL containers (like `queue` and `stack`) are allowed if needed.

You are to create a **templated Heap class in C++**, called `Heap<T>`, representing the **Soul Heap** of the Bleach universe.

Each element represents a **Soul Warrior** with attributes such as:

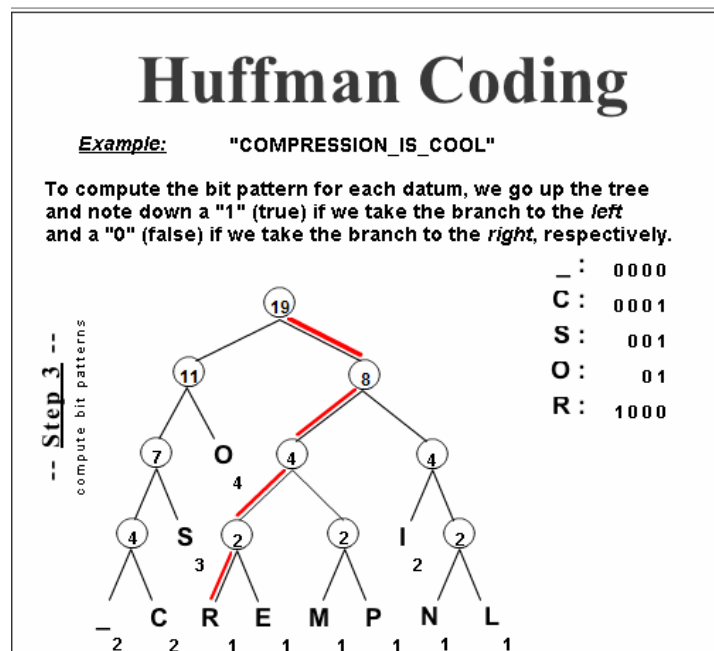
- id: unique Soul identifier
- name: warrior's name (e.g., Ichigo, Rukia, Aizen)
- reiatsu: the warrior's spiritual pressure (int value for heap ordering)

The Heap must maintain its structure through **Reheap Up** and **Reheap Down** operations.

Use the templated Heap and implement a menu to support these functions:

1. Add new warriors
2. Remove the top warrior
3. Display the heap hierarchy
4. Clear the Heap
5. Get Count of Inserted Warriors
6. Increase the Reiatsu of a Warrior
7. Sort the Warriors using the Heap (Heapsort)

Task # 2



Implement the Huffman encoding algorithm to compress a string input by encoding its characters based on their frequencies.

Task Details:

1. Write a function to calculate the frequency of each character in a given string.
2. Use a priority queue (min-heap) to store the nodes, each representing a character and its frequency.
3. Implement the Huffman tree construction by repeatedly extracting the two nodes with the smallest frequencies and combining them.
4. Assign binary codes to each character by traversing the Huffman tree.
5. Encode the input string using the generated Huffman codes.
6. Write a decoding function that takes the encoded string and the Huffman tree to restore the original string.
7. Demonstrate your solution with example inputs and outputs.

Submission Guidelines

1. submit following the only file strictly following the naming convention
 - a. 24L-1234.cpp
-